# PDF to Audiobook

A Brief

Portfolio explainer

## 1 The project in one page

**What it is**

A small desktop application, about 578 lines of Python in two files, that extracts the text from PDF files and converts it into mp3 audiobooks. It handles a batch of files at once, lets you preview the extracted text and restrict conversion to a page range, offers a choice of language and regional accent, reports honest progress while it works, and can play the result. It also runs entirely from the command line with no screen.

The interesting thing about this project is not the feature list. It is that a hobby-scale utility makes several decisions that larger codebases routinely get wrong, and that its README is candid about the ones it did not solve.

### 1.1 The three constraints that shaped it

1. **Speech synthesis happens on Google's servers.** The gTTS library is a thin client for Google Translate's speech endpoint. So the tool needs an internet connection, has no offline mode, is subject to rate limits, and has no volume control because the library offers none. No API key is needed; `.env.example` is deliberately empty and says so.
2. **A PDF is not necessarily text.** A scanned PDF is a picture of a page. This project does no OCR, so such a document yields nothing.
3. **A graphical interface freezes while your code runs.** Any slow work done directly in a button handler locks the window. Conversion is slow by nature. This single fact dictates the architecture of the GUI file.

## 2 Architecture

The project is split in two, and the dependency points one way only.

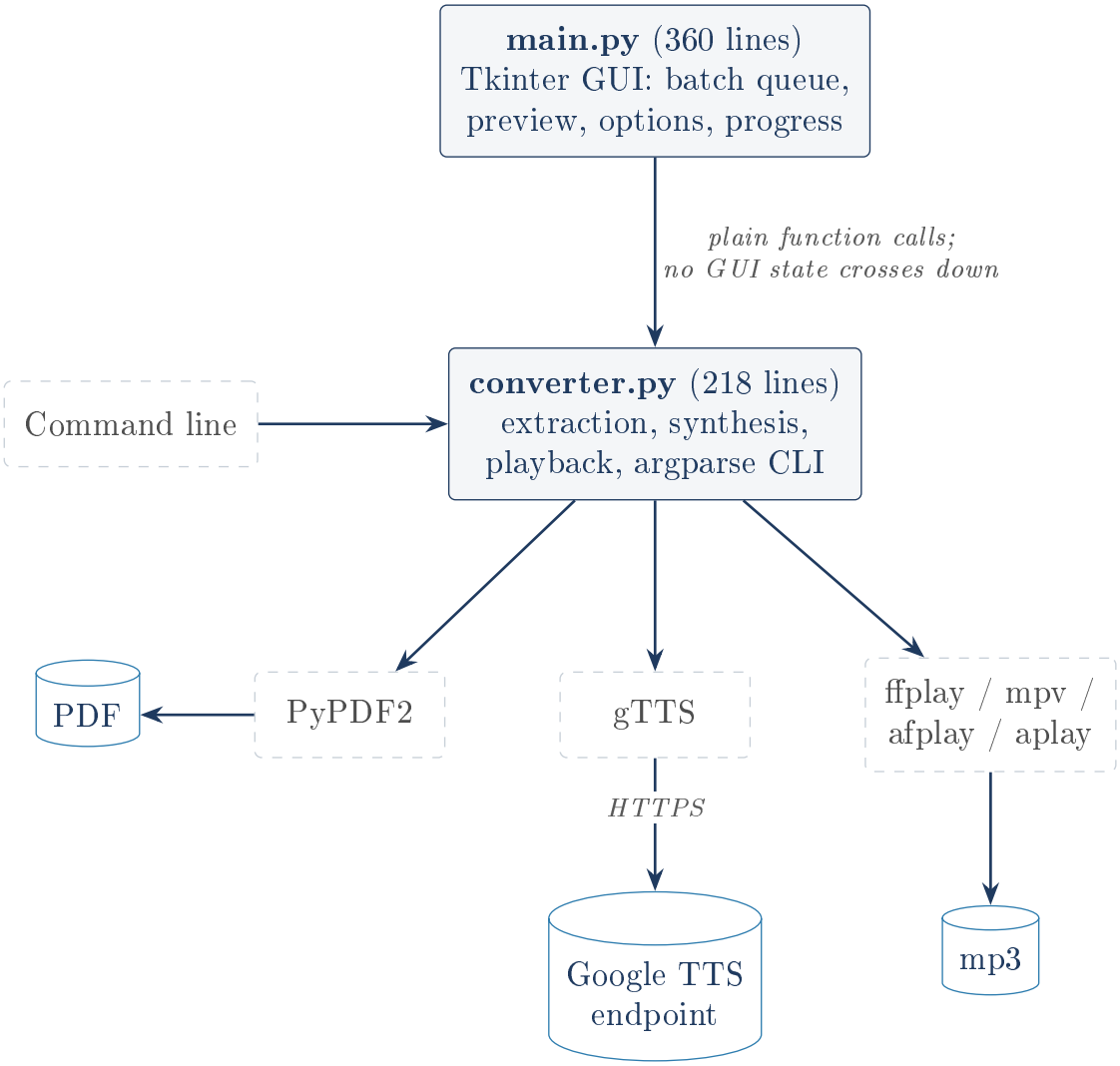


**Figure 1:** `converter.py` never imports Tkinter and never touches a widget. That is what makes both the command line entry point and headless verification possible.

**Why the split is the load-bearing decision**

Because the logic module has no GUI dependency, it can be run and checked without a screen. The README records that an earlier version was a single file mixing widgets and conversion, and that separating them was a deliberate fix. Everything verified in Section 6 of this document was verified by calling those functions directly, which would have been impossible in the original design.

# 3 The two ideas worth understanding

## 3.1 Per-page synthesis is what makes progress real

The obvious implementation sends the whole document to gTTS in one request. That request is a black box: it takes a minute and reports nothing, so any progress bar drawn around it is an animation on a timer, not information.

This project synthesizes **one page per request** instead. The code regains control between pages, and that is where it calls an optional `progress_callback(done, total)`. The GUI turns that into "Converting page 7/20"; the CLI prints `Page 7/20`; a script can pass nothing at all. The progress is truthful because the boundaries are real. The cost, accepted deliberately, is more

requests and more chances to hit a rate limit.

### 3.2 Joining the pages: raw mp3 concatenation

Per-page synthesis leaves one mp3 fragment per page that must become one file. The code keeps a single file handle open and writes each fragment's bytes into it, one after another.

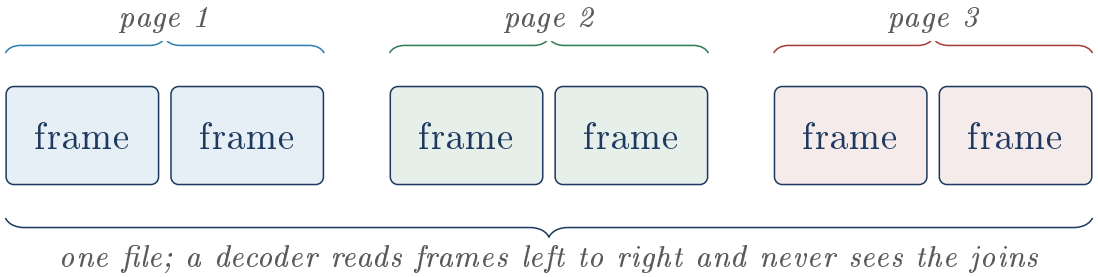


**Figure 2:** An mp3 is a chain of self-describing frames, so gluing streams together works.

This would destroy most formats: concatenating two PNG files produces corruption, not a taller image, because a PNG has one header for the whole picture. An mp3 is a stream of independent frames, each with its own header, so a decoder simply carries on. The README is honest that this leans on a property of the format rather than a documented guarantee, and that a "proper" fix would add a dependency to perform the same byte copy with more ceremony. It was checked here and it holds (Section 6).

## 4 How the interface stays responsive

Tkinter has a hard rule: **widgets may only be touched from the thread that created them**. So conversion cannot run on the main thread (the window would freeze) and cannot update the widgets from the worker thread (undefined behaviour). The project resolves this with the standard three-part pattern.

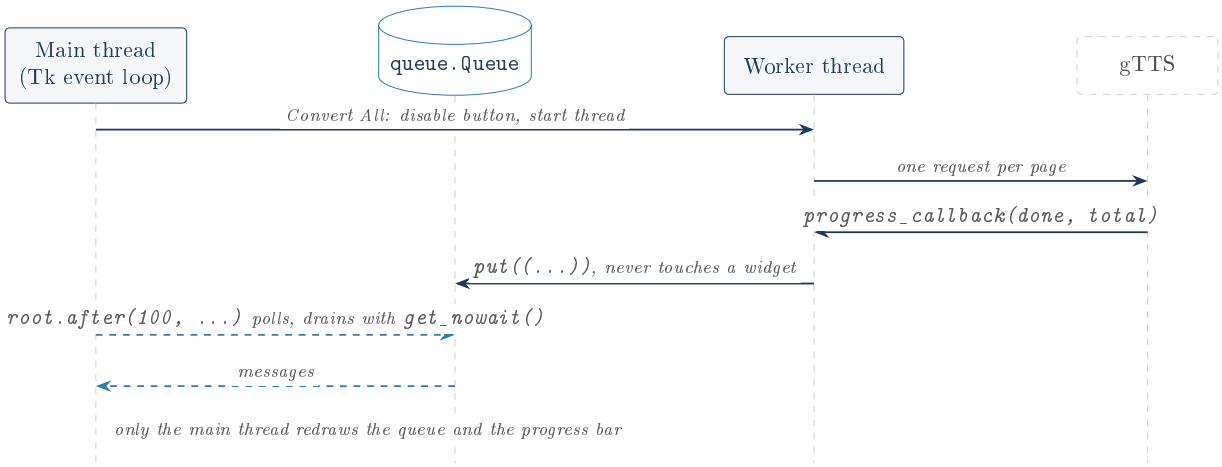


**Figure 3:** The queue gets its own lifeline because no arrow ever runs from the worker straight to a widget. Every fact reaches the display through the queue.

Five message types flow through it: `file_start`, `page_progress`, `file_done`, `file_error` and `all_done`. Each file is wrapped in its own exception handler in the worker, so one bad PDF fails its own row and the batch continues.

## 5 Smaller decisions that show care

- **Atomic writes.** Output goes to `name_tts.mp3.part` and is renamed into place with `os.replace` only on success, with a `try/finally` cleaning up. A conversion interrupted at page 7 of 20 leaves no half-file masquerading as a finished audiobook.
- **Graceful degradation.** `tkinterdnd2` is imported in a `try/except`. If it is missing, drag and drop is skipped, the window's subtitle says so plainly, and the app runs on through the file dialog.
- **No fake controls.** gTTS has no volume parameter, so there is no volume slider. The README says why. A dummy slider would have been easier and worse.
- **Honest failure from `play_audio`.** It searches for `ffplay`, `mpv`, `afplay` and `aplay` in order and returns a boolean, so the GUI can say "no player found, here is the path" instead of doing nothing. The README records that an early draft failed silently here.
- `page.extract_text() or ""`. PyPDF2 returns `None` for a page it cannot read, which would crash the join. One idiom turns a crash into a blank page, and an all-blank range raises a clear error rather than writing an empty mp3.
- **Exact dependency pins** (`PyPDF2==3.0.1`, `gTTS==2.5.4`, `tkinterdnd2==0.6.2`), which is the correct choice for an application.

## 6 What was verified by running it

These claims were not taken from the README on trust. They were re-checked by executing the code against a generated 3-page PDF and a text-free PDF.

| Claim | Result |
|---|---|
| Page-range extraction | Pages 2 to 3 returned exactly those pages, inclusive at both ends. |
| Empty-document handling | Raised the expected `ValueError`; no empty mp3 written. |
| Full conversion | Succeeded; progress fired 1/3, 2/3, 3/3. |
| **mp3 concatenation** | `file` reports a valid `MPEG ADTS, layer III` stream, and `ffmpeg` decoded the whole thing to WAV with no errors. The claim holds. |
| **Page range is real** | 3 pages produced **9.744 s** of audio; pages 2 to 3 produced **6.576 s**. An exact 3:2 ratio, 3.248 s per page, which is strong evidence the joins drop and duplicate nothing. |
| Player discovery | Found `ffplay` first, matching the intended order. |
| GUI freeze bug | Reproduced against a real Tk window. See below. |

`tkinterdnd2` was not installed on the verifying machine, so the drag-and-drop path was not exercised; the documented fallback path is the one that ran.

# 7 Honest findings

**The one serious bug: the interface can be permanently frozen by an ordinary click**

Pressing **Clear Queue** or **Remove Selected** while a conversion is running bricks the window until restart. This was reproduced by running the real code, not inferred.
`start_conversion` disables only the Convert All button; the queue-editing buttons stay live. Queue messages carry a *list position* as the item's identity, and clearing the queue invalidates every position in flight. The next poll raises `IndexError`. The polling function's only `except` catches `queue.Empty`, so the error escapes, and execution never reaches the final line that re-arms the timer via `root.after`. The self-rescheduling chain breaks and never restarts.
Observed: the `IndexError` fires, the queue never drains again, and Convert All stays `disabled` even after `all_done` is posted, because nothing is alive to read it. The worker thread meanwhile finishes and writes its mp3s successfully, which the user cannot learn from the frozen interface.
Either fix suffices: disable the queue-editing buttons during conversion, as Convert All already is; or give messages a stable identity (the file path) instead of a list position.

**The rest, worst first**

- **An inverted or out-of-range page range blames the document.** Verified: requesting pages 3 to 1, or page 99 of a 3-page file, produces "No extractable text found ... for the selected page range". The document is full of text; the range is the problem. A user could conclude their PDF was a scan. The GUI escapes this only because its Save Page Range applies `min`/`max`; the CLI is fully exposed.
- **The CLI prints raw tracebacks at users.** Verified for a missing file and a bad range. A traceback is a developer's diagnostic, not a user message.
- **Silent overwrite.** The output filename derives only from the PDF's name, so converting pages 1 to 5 and then 6 to 10 of one file overwrites the first result without warning. For a tool whose headline feature is page ranges, that is a sharp edge.
- **Progress labels mislead on a range.** "Converting page N/M" counts positions within the selected range, so pages 5 to 7 display "page 1/3".
- `QueueItem` **swallows every exception,** reducing encrypted, corrupt and non-PDF files to an indistinguishable `?` with the real error discarded.
- **Language and accent are independent dropdowns,** so most of the 80 offered combinations (Japanese with an Irish accent) are meaningless, and the accent list contains a duplicate: "Default (.com)" and "US (.com)" are the same value.
- **No automated tests.** There is no test directory. Every check above was written for this document and discarded. The clean module split means a pytest suite over `converter.py` would be easy; its absence is a choice, not an obstacle. The README lists this itself.
- **Offline it does nothing.** Correctly and prominently documented, but still the largest functional limit.

**The fair verdict**

The engineering instincts here are better than the project's size suggests: the one-way module split, the atomic rename, the queue-based thread communication, the boolean- returning player search, and the refusal to ship a fake volume slider are all decisions a careful engineer makes deliberately. The README is unusually honest, including about things this review

independently confirmed.
The weaknesses cluster in one place: the seam between the GUI's data model and the worker thread, where list positions are used as identities and only one of several dangerous buttons is disabled during a conversion. That is where the only severe bug lives, and it is a contained fix rather than a design flaw. Everything else on the list is polish.

# 8 Glossary

**Atomic rename**
A filesystem operation that either completes fully or not at all, with no visible in-between state. Used here so a partial audiobook never appears under the final filename.

**Callback**
A function handed to another function so it can call back while working. Here the converter calls `progress_callback` with a done count and a total after each page.

**Daemon thread**
A background thread that does not keep the program alive when the main window closes.

**Event loop**
The loop a GUI sits in, waiting for clicks and redrawing. While your code runs inside it, nothing redraws, which is why slow work must go elsewhere.

**Graceful degradation**
Losing an optional feature cleanly instead of crashing when something is unavailable.

**mp3 frame**
The unit an mp3 is built from. Each carries its own header, which is why concatenated mp3 streams still decode.

**OCR**
Optical Character Recognition: reading text out of a picture. This project has none, so scanned PDFs yield nothing.

`queue.Queue`
A container designed for safe hand-off between threads. The standard answer to inter-thread communication in Python.

**Thread**
A second line of execution inside one program, running concurrently.

**TLD**
Top-level domain, the end of a web address. gTTS uses it to pick a regional Google endpoint, which is how the "accent" setting works.

**TTS**
Text to Speech. Here it is a network service, not a local engine.